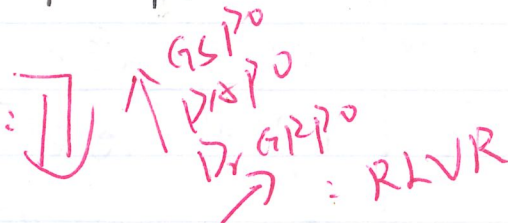


Lecture #16: RL algorithms

pre-training + RLHF $\Rightarrow$ GPT3.5, ... today: GPT-ol/r1

core contents for this lecture: 

Core algorithms: PPO $\rightarrow$ GRPO $\rightarrow$ GRPO variants

E.g. Deepseek
math

E: underscoring

RL-zero-like Training

case studies: Deepseek-R1 / KIMI-K1.5 / Gwens tech report
(three Reasoning Models)

Three Attempts to reach PPO:

①: basic ~~gradient~~ policy gradient: push harder when reward is high, but rewards are noisy/random, fast but it is unstable

②: TRPO: push ^{policy} update, but stay in a safe zone, but complicated

③: PPO: improve, but cap how much each decision can change, push policy update, but stop if you push too hard, it is simple and stable

PPO: proximal policy optimization = gradient method for optimizing rewards in actual RL tasks

< The 37 implementations details of PPO >

RL algorithms.

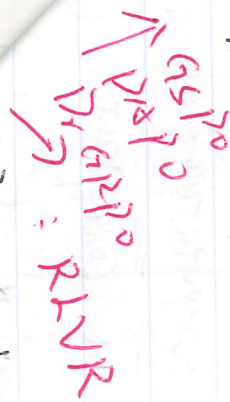
today: GPT-ol/r1.

GPT-ol/r1 variants

From encoding

0-like Training

report



- rollouts
- loss computation
- reward shaping
- add per-token KL
- minimize the loss over
- some rollouts
- pretty standard stuff
- basically all there

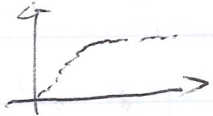
rate

use advantages

what do you expect to see in ppo?

reasonable training curves

①: increasing overall rewards:



②: incline reward model:



③: Negative KL Rewards:

